Processes

What a running program really is, how the OS keeps track of it, how processes are born and die, and how they talk.

Chapter 4 of the notebook

BIG QUESTION

Is a program the same thing as a process?

Today's route

What we'll cover today

1. **Process concept**
   Contents, states, and the PCB
2. **Process scheduling**
   Queues, schedulers, context switch
3. **Operations on processes**
   Creation and termination
4. **Cooperating processes & IPC**
   Message passing and shared memory

Part 01

The process concept

THINK FIRST

A recipe in a book, and a cook actually cooking it. Which one is the program, and which one is the process?

discuss for one minute before the next slide

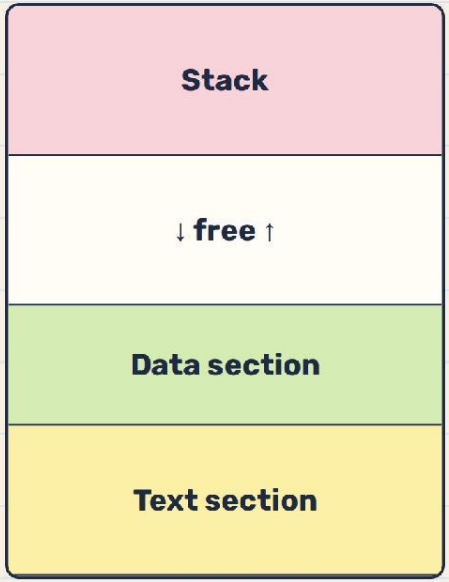
Definition

A process is a program in execution.

The program is the recipe on the page (passive). The process is the cooking in progress (active).

What's inside

A process is more than its code



One process in memory

Text section

The program instructions.

Program counter

Points at the next instruction to run.

Stack

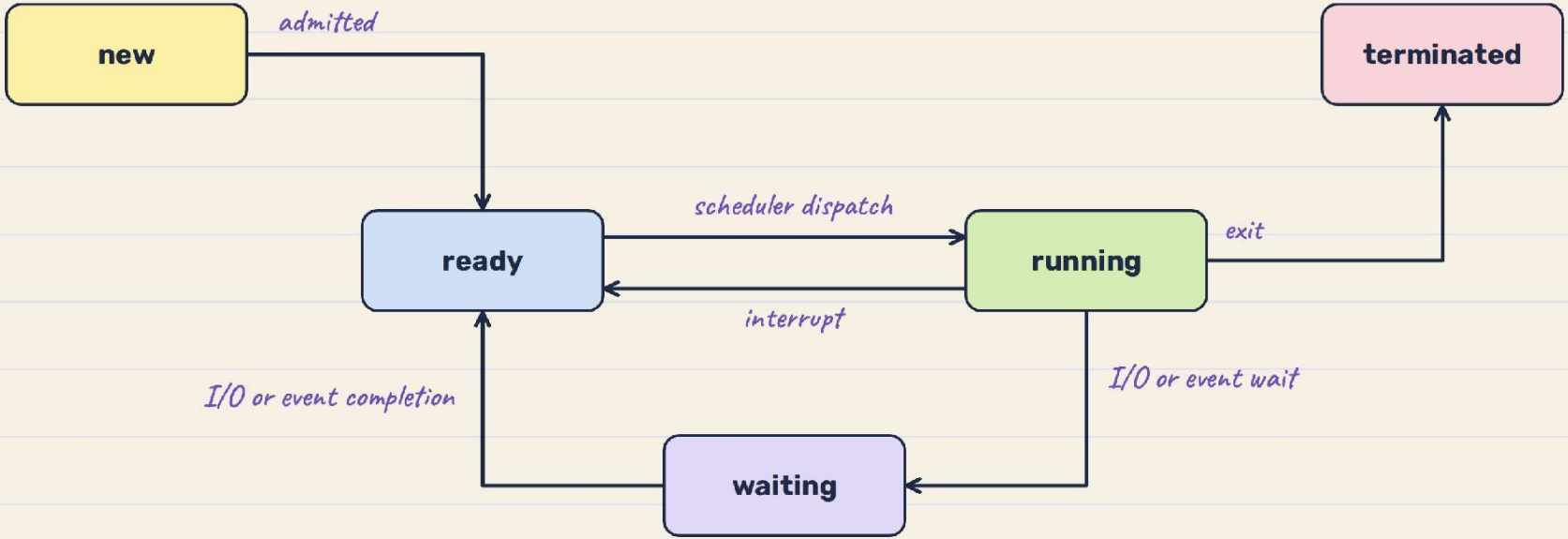
Local variables, return addresses, method parameters.

Data section

Global variables.

Process state

Five states, six transitions



Process state

What each state means

State	Meaning	Bank analogy
new	The process is being created	Walking in the door
ready	Waiting to be assigned a processor	Waiting in line
running	Instructions are being executed	At the teller's window
waiting	Waiting for some event (e.g. I/O)	Stepped aside to fill a form
terminated	Finished execution	Left the bank

Process Control Block

The PCB is the process's ID card

Process state
Process number (ID)
Next instruction to run
Saved CPU work (registers)
Scheduling info (priority)
Memory info
Accounting (CPU time used)
I/O status (open files, devices)



One per process

The kernel keeps a PCB for every process: everything it needs to stop the process and later resume it.



Real life

A patient's file at a clinic: who they are, where they stopped, what they're waiting for.

CPU switch

From process to process, through the OS

Process P0	Operating system	Process P1
executing	–	idle
idle	interrupt → save state into PCB0, reload state from PCB1	idle
idle	–	executing
idle	interrupt → save state into PCB1, reload state from PCB0	idle
executing	–	idle

time flows downward ↓

Context switch

Switching has a price



What happens

The system saves the state of the old process and loads the saved state of the new one.



Pure overhead

While switching, the system does no useful work.



Keep it rare

The more often we switch, the more time is lost to switching itself.

↺ *Chapter 5's dispatcher performs exactly this switch*

Part 02

Process scheduling

THINK FIRST

A bank with one teller. Who decides who gets into the building, and who is served next?

discuss for one minute before the next slide

Why schedule?

Two goals from Chapter 1

↺ Recall · multiprogramming, Ch 1

Some process running at all times, to maximise CPU utilisation.

↺ Recall · time-sharing, Ch 1

Switch the CPU so often that users can interact with every program while it runs.

scheduling is how the OS delivers both

Scheduling queues

Processes migrate between queues



Job queue

All processes in the system.



Ready queue

Processes in main memory, ready and waiting to execute.



Device queues

Processes waiting for a particular I/O device. One queue per device.

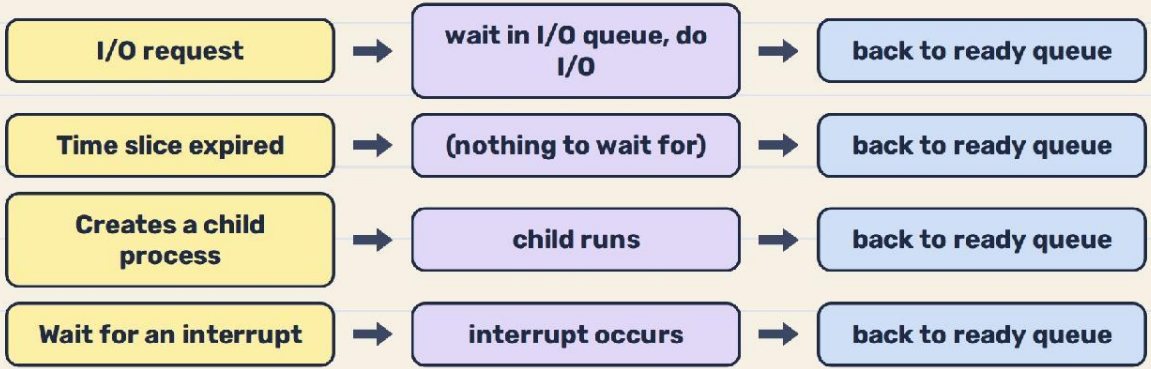
Each queue is a linked list of PCBs, using the pointer field we saw in the PCB.

Queueing diagram

Every path leads back to the ready queue



While on the CPU, one of four things can send a process away:



Schedulers

Three schedulers, three time scales

	Long-term (job)	Short-term (CPU)	Medium-term
Decides	Which processes enter the ready queue	Which ready process runs next	Which process to swap out, then back in
Runs	Infrequently (seconds, minutes)	Very frequently (milliseconds)	Now and then, under memory pressure
Controls	Degree of multiprogramming	CPU allocation	Memory load
Bank analogy	Guard at the door	Teller calling "next!"	"Please step out and come back later"

The long-term scheduler may be absent in some OSes • swapping in detail → Chapter 7

The long-term scheduler's job

Keep a good mix



I/O-bound process

Spends more time doing I/O than computing. Many short CPU bursts. Example: a text editor.



CPU-bound process

Spends more time computing. Few, very long CPU bursts. Example: rendering a video.



Why mix them?

All I/O-bound: the CPU sits idle. All CPU-bound: the devices sit idle. A mix keeps both busy.

→ *Chapter 5 looks at these CPU bursts up close*

Pop quiz • click to reveal

Which scheduler runs most often?

A Long-term scheduler

B Short-term (CPU) scheduler

C Medium-term scheduler

D They all run equally often

Pop quiz · click to reveal

Which scheduler runs most often?

A Long-term scheduler

B Short-term (CPU) scheduler

C Medium-term scheduler

D They all run equally often

Answer B. The CPU scheduler runs every few milliseconds, every time the CPU needs a new process.

Part 03

Operations on processes

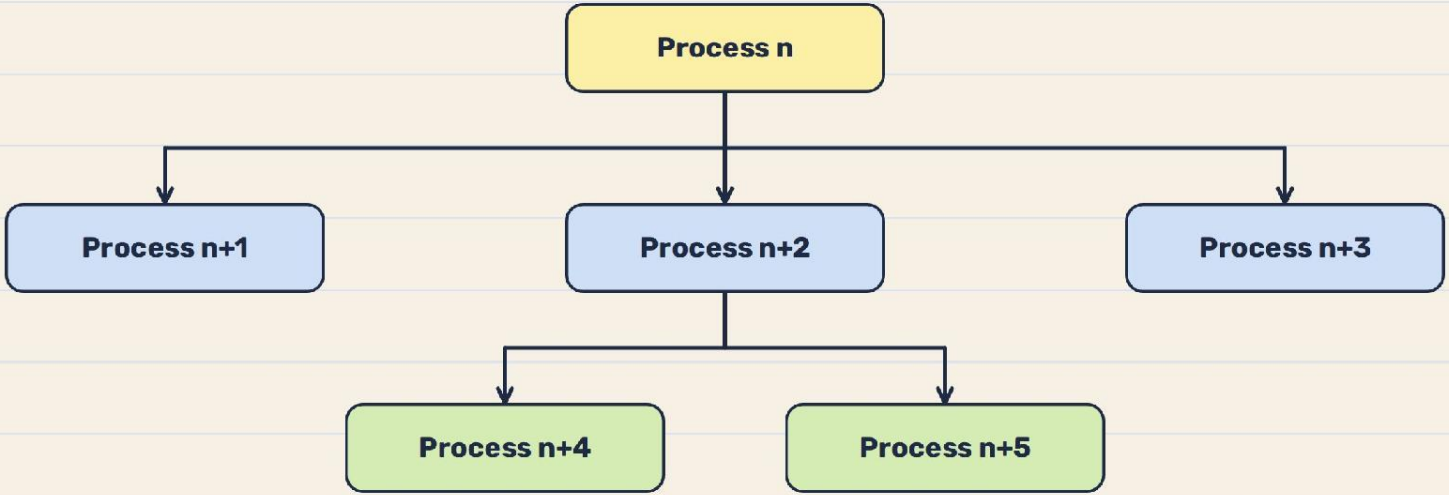
THINK FIRST

When you double-click an app, who creates its process?
Another process!

discuss for one minute before the next slide

Process creation

Parents create children: a tree of processes



the very first process is started by the OS when the computer boots

Process creation

Two design choices



Resource sharing

- Parent and children share all resources
- or children share a subset of the parent's



Execution

- Parent and children run concurrently
- or the parent waits until the children terminate



Real life

A manager (parent) hires assistants (children), may share the office equipment with them, and either works alongside them or waits for their report.

Process termination

Two ways to end



Normal exit

- Process runs its last statement and asks the OS to delete it (**exit**)
- It can hand its results back to its parent
- The OS de-allocates its resources



Abort by the parent

- Child exceeded its allocated resources
- Child's task is no longer required
- Parent is exiting: the OS won't let the child continue → **cascading termination**

Pop quiz • click to reveal

A parent process exits and the OS terminates all of its children too. What is this called?

A Context switch

B Cascading termination

C Swapping

D Spooling

Pop quiz • click to reveal

A parent process exits and the OS terminates all of its children too. What is this called?

- **A** Context switch
- **B** Cascading termination
- **C** Swapping
- **D** Spooling

Answer B. When the parent goes, its children (and their children) are terminated with it.

Part 04

Cooperating processes & IPC

THINK FIRST

Two people writing one report: pass notes back and forth, or both write on the same whiteboard?

discuss for one minute before the next slide

Cooperating processes

Independent vs cooperating



Independent

Cannot affect, or be affected by, the execution of another process.



Cooperating

Can affect, or be affected by, another process.

Information sharing

Computation speed-up

Modularity

Convenience

Inter-process communication

Two models: messages or shared memory

Message passing

Process A sends a message

↓ the OS delivers it ↓

Process B receives it

Shared memory

Process A writes

Shared region (A and B both see it)

Process B reads

↺ Recall · OS services, Ch 3

Communication was listed as a service. This is how it actually works.

Message passing

How the messages travel



Message size

- Fixed size: simpler for the OS
- Variable size: more flexible for programs

Direct

Send to a named process, like calling a person by name.

Indirect

Drop it in a shared mailbox; whoever checks the mailbox picks it up.

Synchronization

Blocking or non-blocking?

	Blocking (synchronous)	Non-blocking (asynchronous)
send	Sender waits until the message is received	Sender sends and carries on
receive	Receiver waits until a message arrives	Receiver gets a message, or nothing, and carries on

↺ Recall • sync vs async I/O, Ch 2

Same idea as synchronous vs asynchronous I/O: wait for it, or carry on.

Pop quiz • click to reveal

With a *non-blocking* send, what does the sender do after sending?

A. Waits until the receiver reads it

B. Carries on immediately

C. Terminates

D. Swaps itself out

Pop quiz • click to reveal

With a *non-blocking* send, what does the sender do after sending?

- **A** Waits until the receiver reads it
- **B** Carries on immediately
- **C** Terminates
- **D** Swaps itself out

Answer B. Non-blocking = asynchronous: the sender continues right away.

Wrap-up

What to remember

OS JOURNAL · CHAPTER 4

- ✓ A process is a program in execution: text, counter, stack, data.
- ✓ The PCB holds everything needed to pause and resume a process.
- ✓ Long-, short- and medium-term schedulers work on different time scales.
- ✓ Five states: new, ready, running, waiting, terminated.
- ✓ A context switch saves one PCB and loads another: pure overhead.
- ✓ IPC: message passing via the kernel, or shared memory.

Next up *Chapter 5 · CPU Scheduling: who runs next, and why*